

SPEC 2111

Deep Learning and Neural Networks

Course Roadmap

Brightspace: SPEC-2111-54148-TU5353-FT-202520

PART-1: Introduction, Foundations & Essentials

- Week 1 – Introduction & Course Overview
- Week 2 – Linear Regression
 - Regression concepts, gradient descent, loss functions
- Week 3 – Logistic Regression
 - Classification, sigmoid, cross-entropy loss

PART-2: Neural Networks, Deep Learning and Training

- Week 4 – Neural Network Essentials
 - Neurons, layers, forward propagation, activation functions 
- Week 5 – Refining Backpropagation and Deep Neural Networks (DNNs)
 - Backpropagation, chain rule, gradient descent optimization, intro to multiple layers
- Week 6 – Preventing Overfitting
 - Overfitting vs underfitting, regularization (L1/L2), dropout, early stopping

PART-3: Specialized Architectures

- Week 7 – Convolutional Neural Networks (CNNs)
 - Convolutions, pooling, image applications
 - Assessment-1
- Week 8 – Representations & Transfer Learning (and Test)
 - Feature learning, pre-trained models, fine-tuning
- Week 9 – Recurrent Neural Networks & LSTM
 - RNNs, LSTM/GRU, sequence modeling
- Week 10 – Attention & Transformers
 - Attention mechanism, transformer architecture, NLP applications

PART-4: Reinforcement Learning & Projects

- Week 11 – Reinforcement Learning
 - Assessment-2
- Week 12 - Project Presentations & Discussions (Project Due)

Week-4 / Agenda

Theme: Neural Network Essentials

- Theory
 - Recap - Week 2&3 on Linear and Logistic Regression
 - Feature Engineering Needs and Neural Networks
 - When Not to Use DL
 - Neurons
 - Network Layers
 - Important Concepts for Neural Networks
 - Single to Multiple Outputs
 - Feed Forward Algorithm
- Lab Work





Main Components of a Learning Algorithm

A learning algorithm consists of:

- A model (with learnable parameters)
- A loss / cost function (measures error)
- An optimization method (updates parameters)



Main Components of Linear Regression

- A model (with learnable parameters)
 - parameters to learn: θ_0, θ_1
- A loss / cost function (measures error)
 - MSE
- An optimization method (updates parameters)
 - Gradient Descent

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=0}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Gradients

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$
$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

Parameter Update

$$\theta_0 := \theta_0 - \alpha \frac{\partial J}{\partial \theta_0}$$
$$\theta_1 := \theta_1 - \alpha \frac{\partial J}{\partial \theta_1}$$



Main Components of a Logistic Regression

- A model (with learnable parameters)
- A loss / cost function (measures error)
 - Cross Entropy Loss
- An optimization method (updates parameters)
 - Gradient Descent

$$h_{\theta}(x) = \frac{1}{1 + e^{-z}} \quad z = \theta x \quad = \sum_{i=0}^n \theta_i x_i$$

$$\text{cost}(h_{\theta}(x^{(i)}), y^{(i)}) = -y^{(i)} \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))$$

Gradients

$$\frac{\partial}{\partial \theta_j} J_{\text{logistic}}(\theta) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Parameter Update

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J_{\text{logistic}}(\theta)$$



The Logistic (Sigmoid) Function

Interpretation:

- $\theta^T x \rightarrow$ linear score
- $g(\cdot) \rightarrow$ converts score to probability

Key idea:

Logistic regression = linear model + sigmoid

$$h_{\theta}(x) = g(\theta^T x)$$



SIGMOID FUNCTION

$$g(x) = \frac{1}{1 + e^{-x}}$$



$$z = \theta^T x$$

LOGIT

- In logistic regression, **logit** (also called pre-activation) is the raw linear score before applying sigmoid to obtain a probability.
- logistic sigmoid is important because it converts logits into probabilities between 0 and 1, making it ideal for the output layer of neural networks in binary classification.



Error Functions

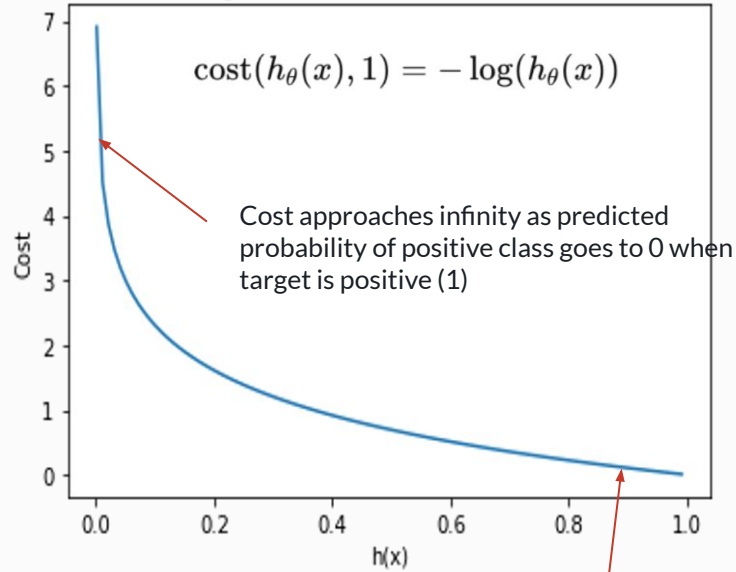
- MSE measures the squared distance between predicted and true values
 - suited for regression
 - does not care about class labels.
- Log loss measures how well predicted probabilities match true class
 - it is suited for classification
 - penalizes confident wrong predictions.

The key difference:

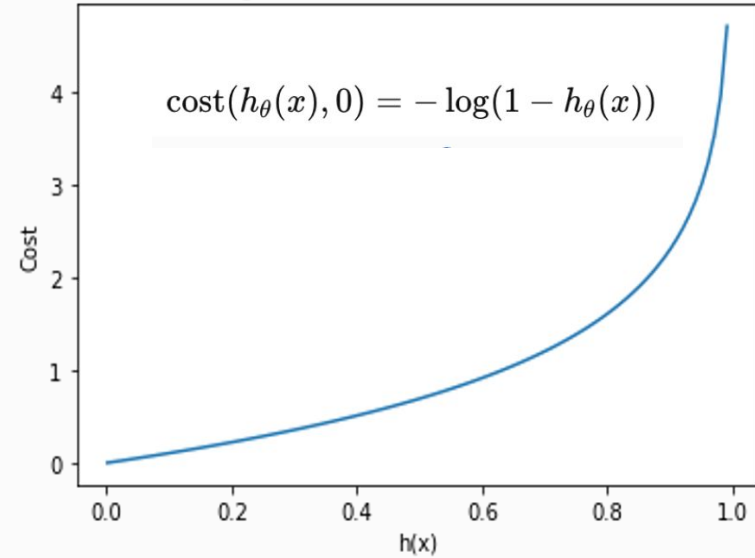
- MSE is distance-based, class-agnostic
- log loss is probability-based, class-sensitive.



Logistic Cost Function where $Y=1$



Logistic Cost Function where $Y=0$



- approaches zero as predicted probability goes to 0
- approaches infinity as probability approaches 1, heavily penalizing confident misclassifications.



Gradient Descent Algorithm

Repeat until convergence

1. **Compute predictions** for all training examples using current parameters:

$$\hat{y}^{(i)} = f(x^{(i)}; \theta)$$

2. **Compute the loss** over all examples:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{loss}(\hat{y}^{(i)}, y^{(i)})$$

3. **Compute gradients** of the loss w.r.t all parameters:

$$\frac{\partial J}{\partial \theta_{ij}^{[l]}}$$

4. **Update parameters:**

$$\theta_{ij}^{[l]} \leftarrow \theta_{ij}^{[l]} - \eta \frac{\partial J}{\partial \theta_{ij}^{[l]}}$$

Neural Network Essentials

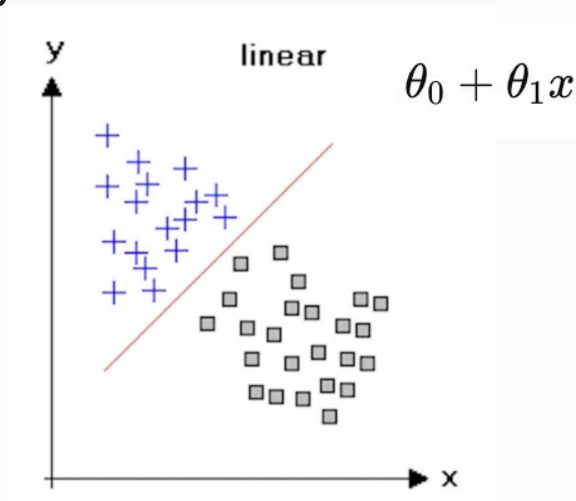
Logistic Regression Learns a Linear Boundary

Suppose you're classifying spam vs not spam pass vs fail, and the data looks like

- Class A: mostly on the left
- Class B: mostly on the right

A straight line cleanly separates them.

Logistic regression works perfectly well in the existing input space



Logistic regression as a model is linear in parameters

- each parameter appears to the first power and is multiplied by a feature
- It never has terms like θ_1^2 or $\theta_1\theta_2$
- The model may include nonlinear features, but the parameters (thetas) themselves always enter linearly.

But you could transform features to another space!

- Inside the circle: $x_1^2 + x_2^2 < r^2$
- Outside the circle: $x_1^2 + x_2^2 > r^2$

No straight line can separate "inside" from "outside."

- But this equation works: $x_1^2 + x_2^2 = r^2$
- If we transform/engineer features, logistic regression can now learn a circular boundary.

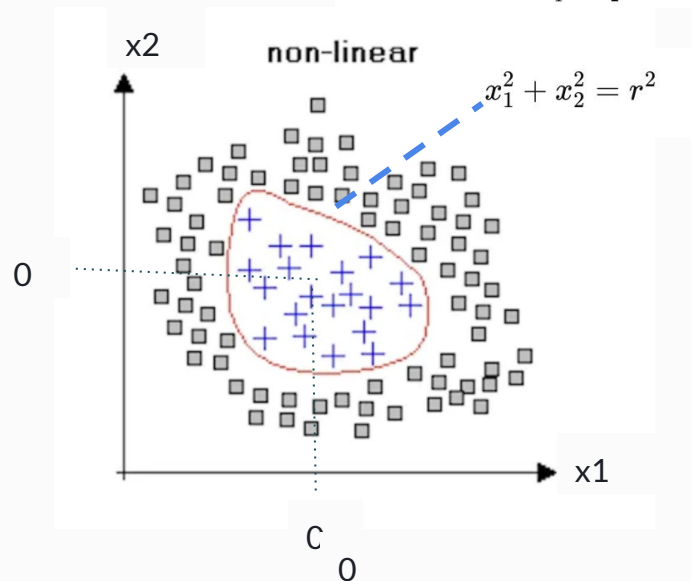
Instead of

$$z = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

Your model becomes

$$z = \theta_0 + \theta_1 x_1^2 + \theta_2 x_2^2$$

Still linear in parameters (thetas!)



Logistic regression works well if the data is linearly separable in the feature space. If the classes are not linearly separable, logistic regression cannot capture the correct boundary unless you transform the features !!

Instead of using x_1 and x_2 directly you engineer a new feature from them

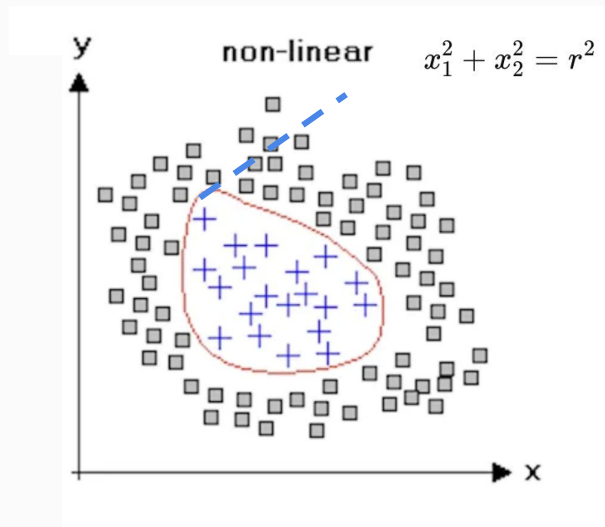
Big conceptual point

The model is always:

$$z = \theta_0 + \theta_1 x_1^2 + \theta_2 x_2^2$$

- Linear in parameters θ
- But nonlinear in inputs

That's the core trick.



What's the problem here?

Hand Crafted Features

- X_1, X_2 = Original Data
- X_3 = squares of X_1
- X_4 = squares of X_2
- X_5 = product of X_1 and X_2
- X_6 = product of X_1 and X_3
-

What's the bottleneck with Logistic Regression?

Hand Crafted Features

- X_1, X_2 = Original Data
- X_3 = squares of X_1
- X_4 = squares of X_2
- X_5 = product of X_1 and X_2
- X_6 = product of X_1 and X_3
- ...

- The number of features grows combinatorially
- Training becomes slow
- Overfitting risk increases
- You must guess which interactions matter

The Bottleneck of linear models is:

you must **manually introduce nonlinearity** through feature transformations to capture complex patterns in data.

But:

$$x_1^2, x_1x_2, x_2^3, \dots$$

- How many terms do we add?
- Which interactions matter?

This becomes impractical if you have more and more features:

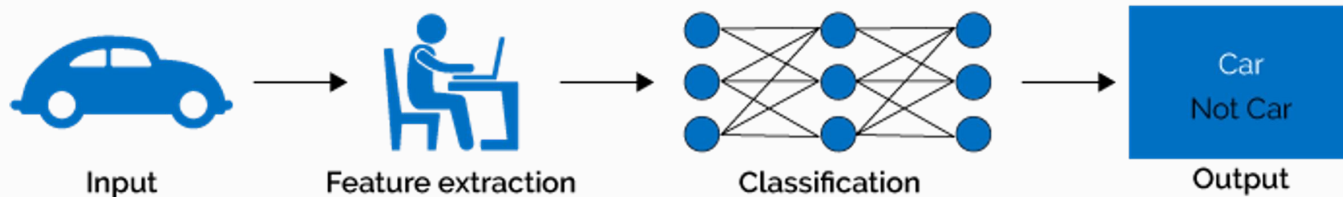
- 2 features → manageable
- 10 features → explosion
- 100 features → impossible

...

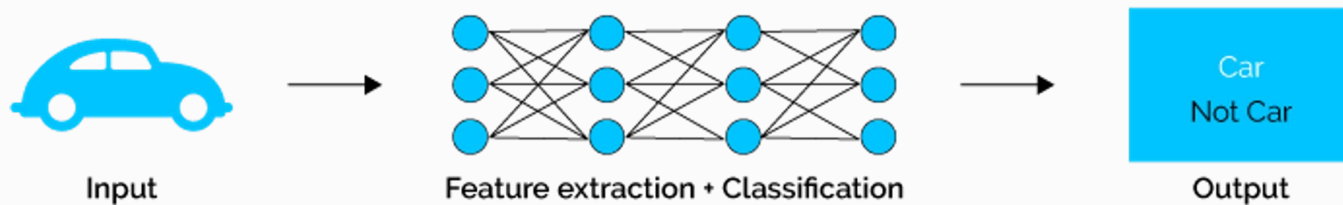
Avoiding Hand Crafted Features

A neural network and deep learning learns internal nonlinear combinations automatically via hidden layers!!

Machine Learning



Deep Learning



However!! before going further, important clarifications

Deep Learning has limitations also and we need to be cautious!

- Does not eliminate all preprocessing
 - Missing value handling, scaling, encoding, sequence/window creation may still be needed
- Not universally better
 - Classical models (Random Forest, XGBoost, linear models) can outperform DL on tabular data
- Requires sufficient data and computational resources to learn effectively

Before going further, important clarifications

Good Use Cases for Deep Learning

- High-dimensional or unstructured data: images, audio, text, video
- Large datasets: hundreds of thousands to millions of labeled examples
- Complex patterns that are hard to define manually
- Sequential or temporal data where relationships over time matter

When to Be Cautious / Classical Models Might Be Better

- Structured/tabular data with clear features
- Small datasets (e.g., <50k–100k labeled rows)
- Highly imbalanced rare events with few positive examples (DL can easily overfit)
- Problems where feature engineering already captures most signals

Key takeaways

- Deep learning does not automatically remove preprocessing or feature engineering
- Classical models (Random Forest, gradient boosting, logistic regression) can outperform DL on small or structured datasets
- DL is a tool — use it when the problem complexity and data size justify it

We will learn key concepts and essentials of Neural Networks



THIS WEEK

- **Logits (pre-activations)**
 - **Preactivation outputs** ($z = Wx + b$)
- **Activation functions**
 - Ex: sigmoid, also later ReLU, softmax, etc.
 - Introduce non-linearity!
- **Layers** (input, hidden, output) and **number of neurons per layer**
 - number of layers (depth)
 - neurons per layer (width)
 - Effects capacity and expressiveness of networks
- **Parameters of the network** (Weights and biases)
- **Forward propagation** (computing outputs from inputs)

NEXT WEEK(S)

- **Backpropagation** (computing gradients via chain rule)
- **Gradient descent / parameter updates**
- **Loss functions** (MSE, cross-entropy, etc.)
- **Hyperparameters** (learning rate, batch size, epochs)
- **Overfitting and regularization** (dropout, L2, early stopping)

We will learn key concepts and essentials of Neural Networks

NEXT WEEK(S)



THIS WEEK

- **Logits (pre-activations)**
 - **Preactivation outputs** ($z = Wx + b$)
- **Activation functions**
 - Ex: sigmoid, also later ReLU, softmax, etc.
 - Introduce non-linearity!
- **Layers** (input, hidden, output) and **number of neurons per layer**
 - number of layers (depth)
 - neurons per layer (width)
 - Effects capacity and expressiveness of networks
- **Parameters of the network** (Weights and biases)
- **Forward propagation** (computing outputs from inputs)

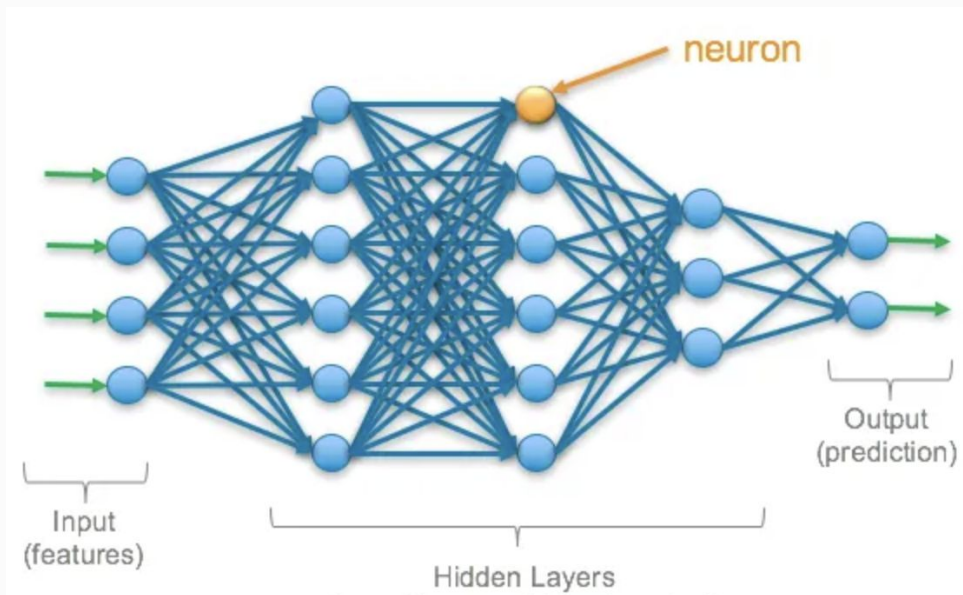
- **Backpropagation** (computing gradients via chain rule)
- **Gradient descent / parameter updates**
- **Loss functions** (MSE, cross-entropy, etc.)
- **Hyperparameters** (learning rate, batch size, epochs)
- **Overfitting and regularization** (dropout, L2, early stopping)

Parameters vs Hyperparameters ??

- values **learned by the model during training** vs. values **set before training**; not learned from data!
- You cannot tune parameters but you can tune hyperparameters!!
- Q: What are parameters and hyperparameters of logistic regression?

From Logistic Regression to Neural Networks

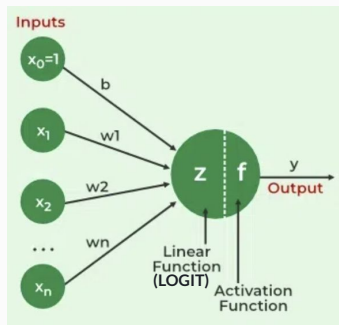
- Neural networks are composed of simple computational units called **neurons**.
- Although inspired by biology, artificial neurons are purely mathematical functions.
- Individually:
 - Simple
 - Limited expressive power
- Combined in layers:
 - Extremely powerful
 - Can model highly complex nonlinear relationships



Mathematical View of a Neuron

A neuron has **three key components**:

1. **Inputs (x)**
 - Can be raw features from data or outputs from other neurons in previous layers.
 - these are the values that are fed into the neuron
2. **Weights and Bias (w,b)**
3. **Activation Function (σ)**



A neuron computes:

$$z = \sum_{i=1}^n w_i x_i + b$$

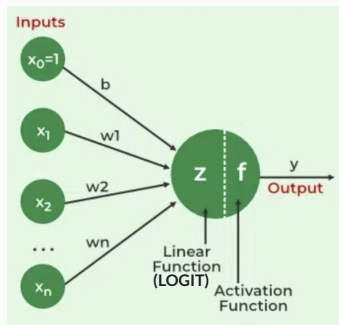
$$a = \sigma(z)$$

- x_i = input features (or outputs from previous neurons)
- w_i = weights (importance of each input)
- b = bias (shifts the activation)
- z = linear combination of inputs (**logit**)
- $\sigma(z)$ = **activation function**, introducing nonlinearity
- a = output of the neuron

Mathematical View of a Neuron

A neuron has **three key components**:

1. **Inputs (x)**
2. **Weights and Bias (w,b)**
 - Model parameters
 - **Weights** determine how much each input contributes.
 - each weight can be interpreted as the relative strength (positive or negative) that we assign to an individual input value x_i
 - **Bias** allows neuron to shift activation function, enabling flexibility in fitting data.
 - Without bias output is always 0 when inputs are 0. Neuron then can't adjust its output when the input is zero.
 - Without bias, it can't 'shift' its activation to fit the data properly.
3. **Activation Function (σ)**



A neuron computes:

$$z = \sum_{i=1}^n w_i x_i + b$$

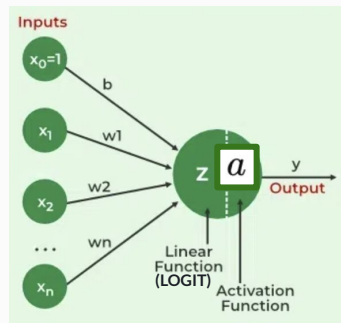
$$a = \sigma(z)$$

- x_i = input features (or outputs from previous neurons)
- w_i = weights (importance of each input)
- b = bias (shifts the activation)
- z = linear combination of inputs (**logit**)
- $\sigma(z)$ = **activation function**, introducing nonlinearity
- a = output of the neuron

Mathematical View of a Neuron

A neuron has **three key components**:

1. **Inputs (x)**
2. **Weights and Bias (w,b)**
3. **Activation Function (σ)**
 - Introduces **nonlinearity**, allowing the network to model complex, nonlinear patterns.



A neuron computes:

$$z = \sum_{i=1}^n w_i x_i + b$$

$$a = \sigma(z)$$

- x_i = input features (or outputs from previous neurons)
- w_i = weights (importance of each input)
- b = bias (shifts the activation)
- z = linear combination of inputs (**logit**)
- $\sigma(z)$ = **activation function**, introducing nonlinearity

Rewrite bias as an input

Original neuron equation:

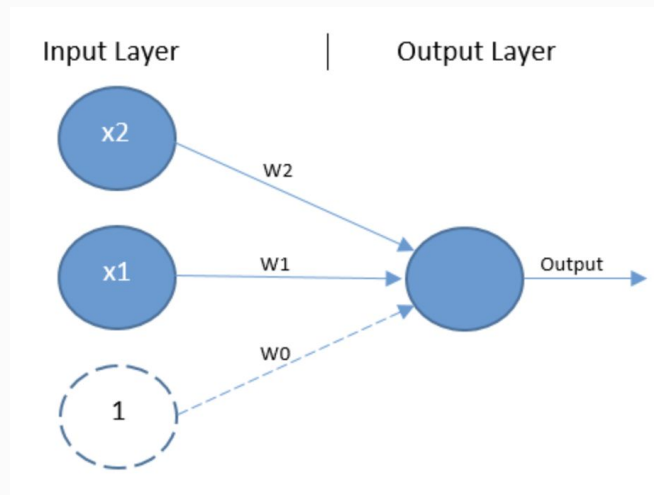
$$z = \sum_{i=1}^n w_i x_i + b$$

Introduce a **dummy input** $x_0 = 1$ and define $w_0 = b$. Then:

$$z = w_0 x_0 + \sum_{i=1}^n w_i x_i = \sum_{i=0}^n w_i x_i$$

Intuition

- Treating b as $w_0 x_0$ lets us **think of the bias as a fixed input feature**.
- The network can now "learn" the bias the same way it learns all other weights.



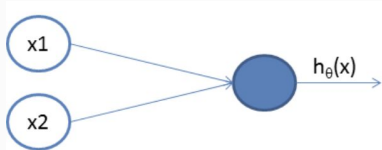
Types of Neurons

- There are many different types of neurons.
 - Some are fully connected, some connected to a partial region in the network
 - Batch-normalized neurons (normalize pre-activations before activation)
 - ..
- But the most significant difference between neuron types lies in their activation function, which determines how logit is transformed into an output.
- For example, a linear activation function produces a continuous real-valued output, which is useful in regression settings.
- Other activations, like sigmoid or ReLU, introduce nonlinearity, which allows the network to learn more complex patterns

Logistic Unit (Sigmoid Neuron)

- A sigmoid neuron is just a neuron that uses the sigmoid activation function
- Logistic regression with a sigmoid is literally a single sigmoid neuron!
- Sigmoid neuron outputs a probability between 0 and 1

$$h_{\theta}(x) = P(Y = 1 \mid x, \theta)$$



LOGIT

$$z = \theta^T x$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-z}}$$

ACTIVATION VALUE

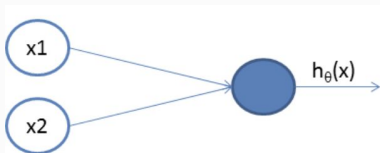
Logistic regression → one logistic unit

Neural network → many logistic units arranged in layers

Sigmoid Neuron - Interpretation

- A sigmoid neuron is just a neuron that uses the sigmoid activation function

$$h_{\theta}(x) = P(Y = 1 \mid x, \theta)$$



$h_{\theta}(x)$ = predicted probability that $y = 1$ given x

$h_{\theta}(x) \geq 0.5 \Rightarrow$ predict class 1

$h_{\theta}(x) < 0.5 \Rightarrow$ predict class 0

$$z = \theta^T x$$

LOGIT

$$h_{\theta}(x) = \frac{1}{1 + e^{-z}}$$

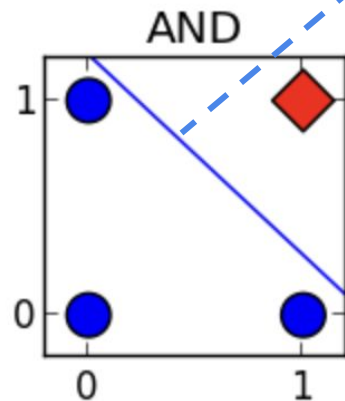
ACTIVATION VALUE

Combining Logistic Units

- Single logistic units or single neurons can only model linearly separable functions.
- However, multiple units can be combined to build more complex functions.
- Let's explore this with logical operations
 - Let's see how combining multiple logistic (sigmoid) neurons can be far more powerful than using them individually!

Implement Logical AND

x1	x2	x1 AND x2
0	0	0
0	1	0
1	0	0
1	1	1



TOY EXAMPLE

- 2 binary inputs, x_1 , x_2
- Bias term, $x_0=1$
- 1 binary output to learn, $y = (x_1 \text{ and } x_2)$
- We are trying to learn this linear decision boundary

AND operator is linearly separable because a single straight line can separate the positive point (1,1) from three negative points in the input space.

Therefore a single sigmoid neuron is sufficient to implement it!

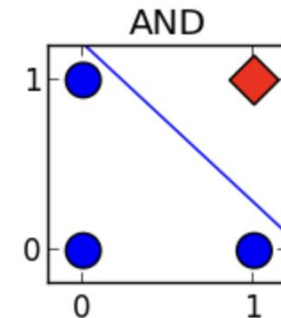
Implement Logical AND

x1	x2	x1 AND x2
0	0	0
0	1	0
1	0	0
1	1	1

$$h_{\theta}(x) = P(Y = 1 \mid x, \theta)$$

$$z = \theta^T x$$

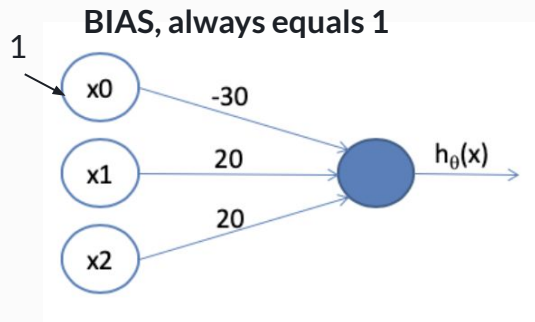
$$h_{\theta}(x) = \frac{1}{1 + e^{-z}}$$



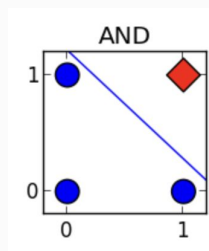
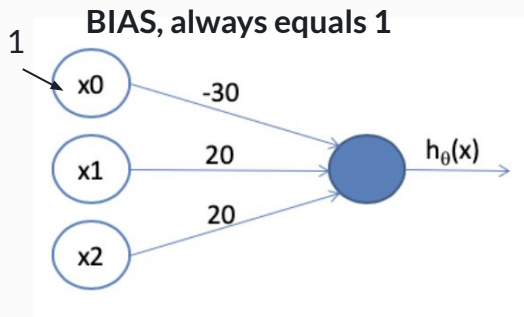
We will use logistic sigmoid unit with a **chosen decision threshold: 0.5**

We will use some **handcrafted weights**:

- We choose weights so that logit is positive only for the desired input pattern.
 - If logit is pos, sigmoid is always > 0.5
- The exact numbers are not unique — any values satisfying the inequalities work.



Implement Logical AND



$$h(z) = \text{sigmoid}(z)$$

$$z = w_0x_0 + w_1x_1 + w_2x_2$$

Case 1: $x_1 = 0, x_2 = 0$

$$z = (-30)(1) + 20(0) + 20(0) = -30$$

$$\sigma(z) = \sigma(-30) \approx 9.36 \times 10^{-14} \ll 0.5$$

Output = 0

Case 2: $x_1 = 0, x_2 = 1$

$$z = (-30)(1) + 20(0) + 20(1) = -10$$

$$\sigma(z) = \sigma(-10) \approx 4.54 \times 10^{-5} < 0.5$$

Output = 0

Case 3: $x_1 = 1, x_2 = 0$

$$z = (-30)(1) + 20(1) + 20(0) = -10$$

$$\sigma(z) = \sigma(-10) \approx 4.54 \times 10^{-5} < 0.5$$

Output = 0

Case 4: $x_1 = 1, x_2 = 1$

$$z = (-30)(1) + 20(1) + 20(1) = 10$$

$$\sigma(z) = \sigma(10) \approx 0.99995 > 0.5$$

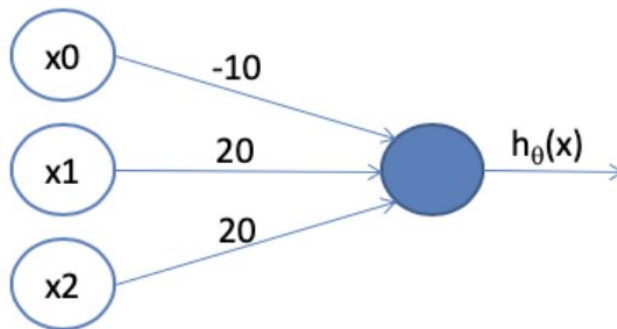
Output = 1

A Single Neuron can model AND operation

Implement Logical OR

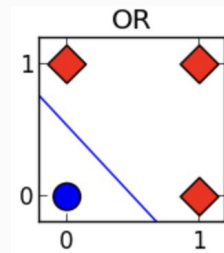
x1	x2	x1 OR x2
0	0	0
0	1	1
1	0	1
1	1	1

BIAS, always equals 1



$$h(z) = \text{sigmoid}(z)$$

$$z = w_0x_0 + w_1x_1 + w_2x_2$$



Case 1: $x_1 = 0, x_2 = 0$

$$z = (-10)(1) + 20(0) + 20(0) = -10$$

$$\sigma(z) = \sigma(-10) \approx 4.54 \times 10^{-5} < 0.5$$

Output = 0

Case 2: $x_1 = 0, x_2 = 1$

$$z = (-10)(1) + 20(0) + 20(1) = 10$$

$$\sigma(z) = \sigma(10) \approx 0.99995 > 0.5$$

Output = 1

Case 3: $x_1 = 1, x_2 = 0$

$$z = (-10)(1) + 20(1) + 20(0) = 10$$

$$\sigma(z) = \sigma(10) \approx 0.99995 > 0.5$$

Output = 1

Case 4: $x_1 = 1, x_2 = 1$

$$z = (-10)(1) + 20(1) + 20(1) = 30$$

$$\sigma(z) = \sigma(30) \approx 1.0 > 0.5$$

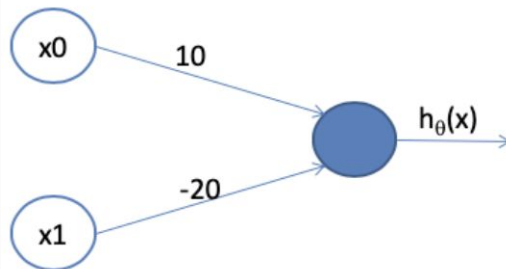
Output = 1

A Single Neuron can model OR operation

Implement Logical NOT

x1	NOT x1
0	1
1	0

BIAS, always equals 1



$$h(z) = \text{sigmoid}(z)$$

$$z = w_0x_0 + w_1x_1 + w_2x_2$$

Case 1: $x_1 = 0$

$$z = 10(1) + (-20)(0) = 10$$

$$\sigma(z) = \sigma(10) \approx 0.99995 > 0.5$$

Output = 1

Case 2: $x_1 = 1$

$$z = 10(1) + (-20)(1) = -10$$

$$\sigma(z) = \sigma(-10) \approx 4.54 \times 10^{-5} < 0.5$$

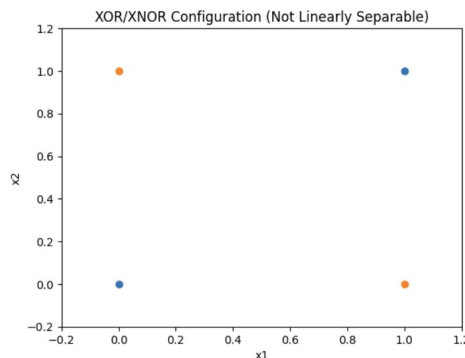
Output = 0

A Single Neuron can model NOT operation

How we can implement XNOR?

Given the AND, OR, and NOT units defined above we can implement more sophisticated functions based on those units.

x1	x2	x1 XNOR x2
0	0	1
0	1	0
1	0	0
1	1	1

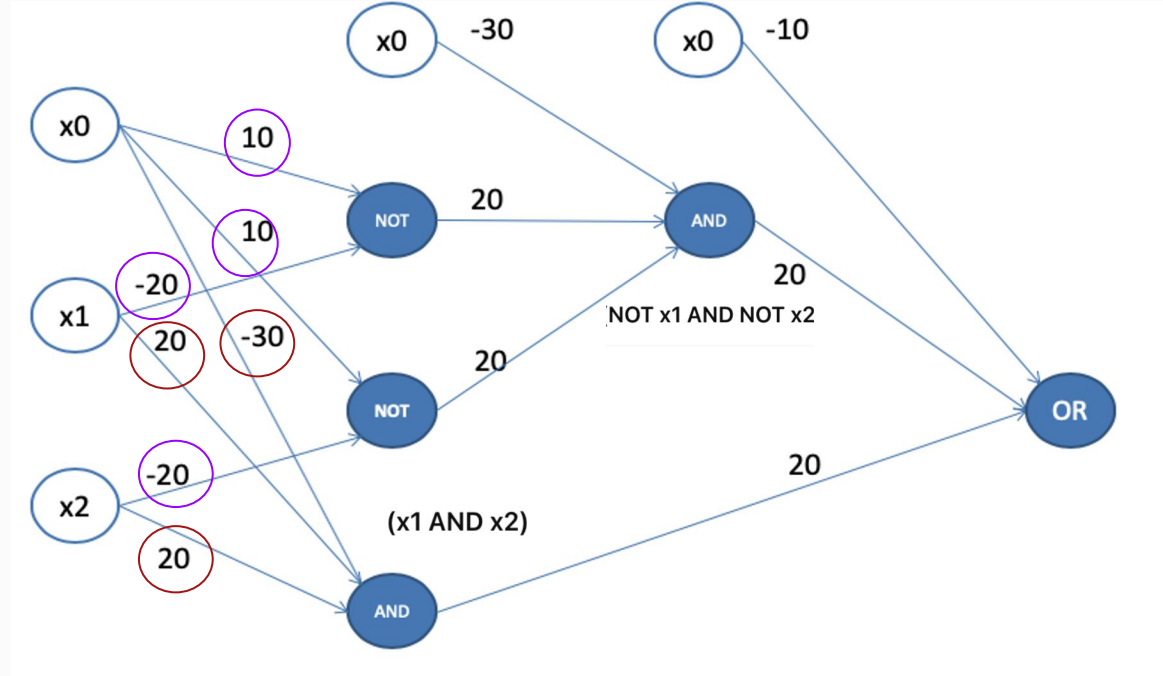
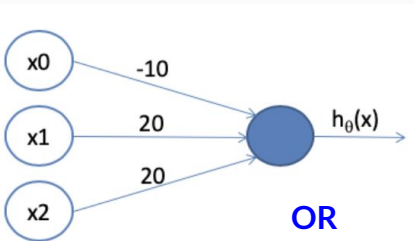
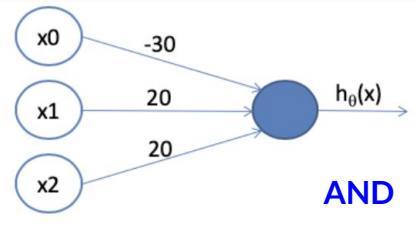
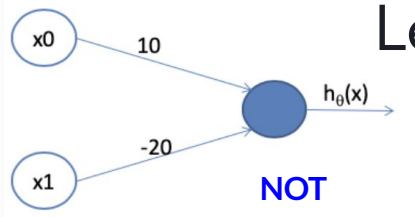


Any straight line will always leave one point from each class on the same side.

$$\text{XNOR} = (\text{x1 AND x2}) \text{ OR } (\text{NOT x1 AND NOT x2})$$

- To correctly classify XNOR, you need a curve or combination of lines.
- This is why a single neuron (linear decision boundary) fails.
- A multi-layer network can combine simpler neurons (AND, OR, NOT) to form this non-linear boundary.

Lets implement XNOR connection multiple neurons



$$\text{XNOR} = (x_1 \text{ AND } x_2) \text{ OR } (\text{NOT } x_1 \text{ AND NOT } x_2)$$

Key Takeaways

- We can use logistic units to construct simple binary functions such as AND, OR, and NOT.
- By layering these units on top of one another, we can combine simple functions to build more complex ones.
 - with the XNOR network we learned the nonlinear decision boundaries automatically, without manually transforming features.
- As we add more layers and more units, neural networks become capable of representing increasingly complex functions
 - complex enough to recognize images, understand speech, and process language.
- XNOR example we built was handcrafted.
 - In practice, we do not manually design these weights.
 - Instead, we use learning algorithms to automatically discover the appropriate weights from data.

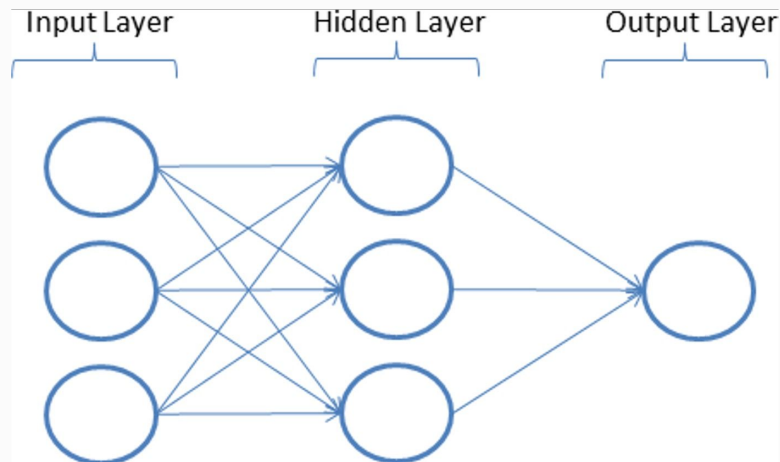
How We Connect Neurons to Build Networks

Neural Network Architectures

- Neural networks are built by connecting logistic units (or other types of neurons) in layers.
 - XNOR example illustrates many key principles of standard neural networks.
- There are **different types of neural network architectures**, each designed for specific kinds of data or tasks, such as:
 - **Feedforward Neural Networks (FFNNs)** – forward-only, layered networks for general tasks
 - No loops, No going back to earlier layers, No time connections
 - **Convolutional Neural Networks (CNNs)** – for grid-like data such as images
 - **Recurrent Neural Networks (RNNs)** – for sequential data like time series or text
 - **Transformer Networks** – for complex sequence modeling, especially in NLP
- We will first focus on the **Feedforward Neural Network (FFNN)** !

Feedforward Neural Network Architecture

- **Input Layer:** Receives the input data vector x .
- **Hidden Layers:** Encode higher-order features and transformations of the input.
 - One or more hidden layers can be found between the input and output layers.
 - Hidden layers encode higher order features which operate over the input data
- **Output Layer:** Produces network's final prediction y , captures the result of our model.

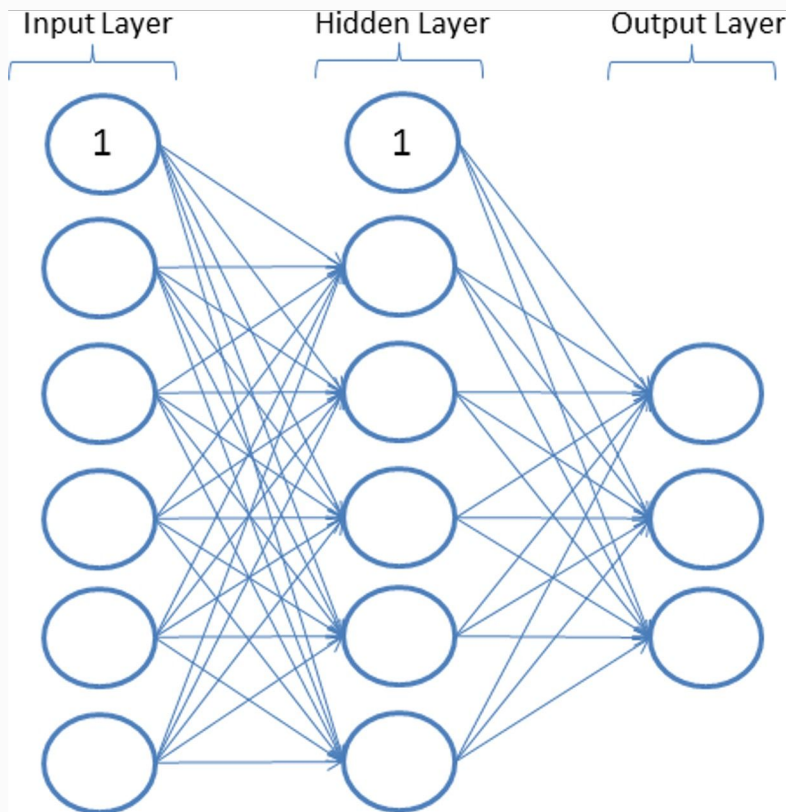


Important Properties of FFNN

- In a typical fully connected FFNN, each node in layer j is connected to all nodes in the previous layer $j-1$ through weights.
- input layer is an exception: it has no incoming connections and simply provides the input data to the network.
- Weights function like the parameters in linear or logistic regression, controlling how input signals are transformed as they propagate through the network.

The Bias Unit

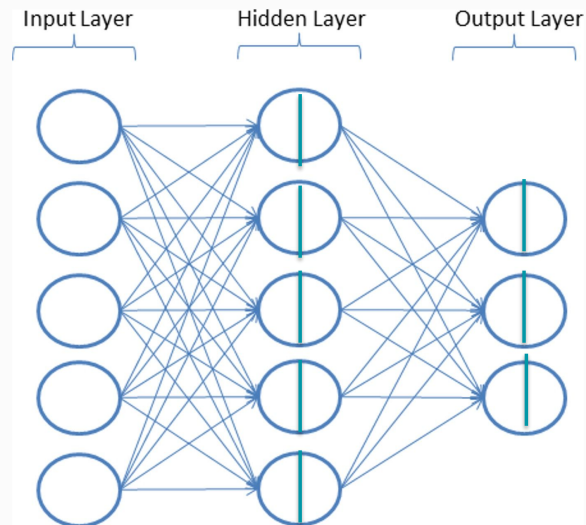
- Each layer except the output layer typically has a bias unit.
- Bias unit in layer j is not connected to units in the previous layer $j-1$!!
 - instead, it provides a fixed input of 1 to the units in the next layer.
 - Bias unit 'data' always 1
- Bias allows the neurons to shift their activation function, analogous to the intercept term in linear or logistic regression.
- Bias is often not visualised in networks!



From Single to Multiple Targets

- One vs All Approach -

- For multi-class problems, output layer can have multiple neurons.
- There are two ways to implement multi-class classification in a neural network:
 - using softmax at the output layer as the activation function
 - using multiple sigmoid outputs (One-vs-All approach).
- If classes are not mutually exclusive (multi-label), use sigmoid at the output layer.
- If classes are mutually exclusive (one class only), use softmax.



One vs All with Sigmoid - Softmax

Method 1: Softmax (Standard Multi-Class)

- One neuron per class at the outer layer with softmax activation
- Probabilities sum to 1
- Classes compete!
- Best when only ONE class is correct
- This is the most common approach

Example:

Cat	Dog	Horse
0.05	0.90	0.05

Interpretation:

- Dog ✓
- Only one class is predicted

Method 2: Sigmoid (One-vs-All Style)

- One neuron per class
- Sigmoid activation per neuron
- Outputs are independent
- This treats each class as: Class i vs Not Class i
- You can still pick the highest output, but probabilities don't sum to 1

Example:

Dog	Cat	Car
0.8	0.7	0.1

Interpretation:

- Dog ✓
- Cat ✓
- Car ✗

This is perfect for images that can contain multiple objects or multi-tag problems.

Aside: Notation for Data

$$Data_{Training} = \{(x^0, y^0), (x^1, y^1), (x^2, y^2) \dots (x^{m-1}, y^{m-1})\}$$

- For each individual training case we assume vectorized inputs and outputs.
- For input data:

- **Superscript** on x^i (or y^i) denotes the **training sample index**.
 - Example: $x^0, x^1, \dots, x^{m-1}$ are the m training examples.
- **Subscript** denotes the **specific feature/component of that sample**.
 - Example: x_2^i is the **third feature** (counting from 0) of the i -th training example.

Suppose:

- $m = 3$ examples
- $n = 4$ features

Then:

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} & x_4^{(1)} \\ x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & x_4^{(2)} \\ x_1^{(3)} & x_2^{(3)} & x_3^{(3)} & x_4^{(3)} \end{bmatrix}$$

Notation for Nodes

- Length of an input vector must be equal to the number of units/neurons on the input layer (not counting the bias unit!).
- Similarly length of an output vector must be equal to the number of units on the output layer of the network.
- for a network with 3 input units and 2 output unit $x^i \in \mathbb{R}^{3 \times 1}, \quad y^i \in \mathbb{R}^{2 \times 1}$
- regression vs classification
 - How many neurons in output layer?

Notation for Layers

- A neural network is built from multiple layers.
- Each layer contains:
 - Nodes (neurons)
 - Associated weights connecting to the next layer

Notation

- L → total number of layers depth
- s_l → number of nodes in layer l (**excluding bias units**) width

Layer Indexing

- Many sources index the first layer as **1**, but we follow **computer science convention**:
 - **Layer 0** → input layer
 - **Layer 1 ... L-2** → hidden layers
 - **Layer L-1** → output layer

Notation for Nodes/Activation Units

For any neuron i in layer j , the computation happens in 2 steps:

Step 1: Weighted Sum (Pre-Activation)

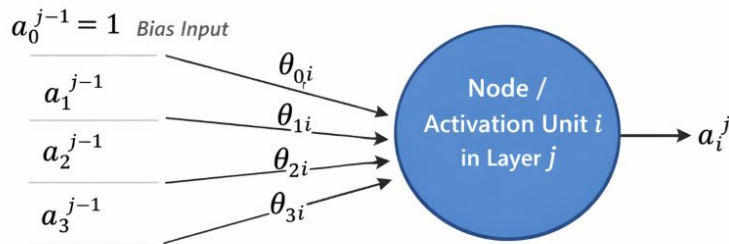
$$z_i^j = \sum_k \theta_{ki}^{j-1} a_k^{j-1}$$

- This is just multiply each input by its weight, k then add them up.
- This value is called the **pre-activation** because it is computed **before** applying the activation function.
- The output value is also referred to as **logit**!

Step 2: Activation

$$a_i^j = g(z_i^j)$$

Now we apply an activation function g (e.g., sigmoid/logistic) to get the final output of the neuron.

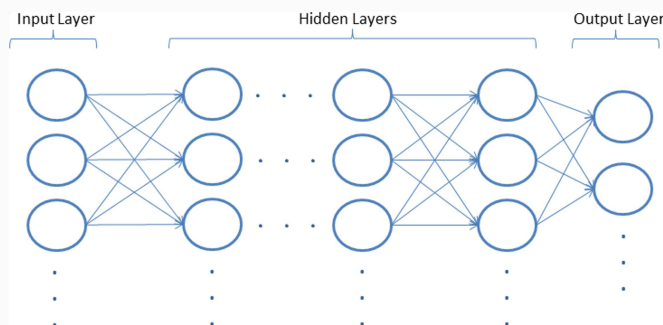


$$\underbrace{z_i^j = \sum_k \theta_{ki}^{j-1} a_k^{j-1}}_{\text{Weighted Sum}} \longrightarrow \underbrace{a_i^j = g(z_i^j)}_{\text{Activation Output}}$$

Weighted sum $\rightarrow z_i^j \rightarrow$ Activation function $\rightarrow a_i^j$

Aside: Notation for Nodes/Activation Units

- Each layer contains activation units (neurons).
- We usually index the output values (activations) rather than the neurons themselves.
- The output of a node = activation value.



Superscript $\rightarrow$ layer index (0-based)

Subscript $\rightarrow$ index of the node within layer (0-based)

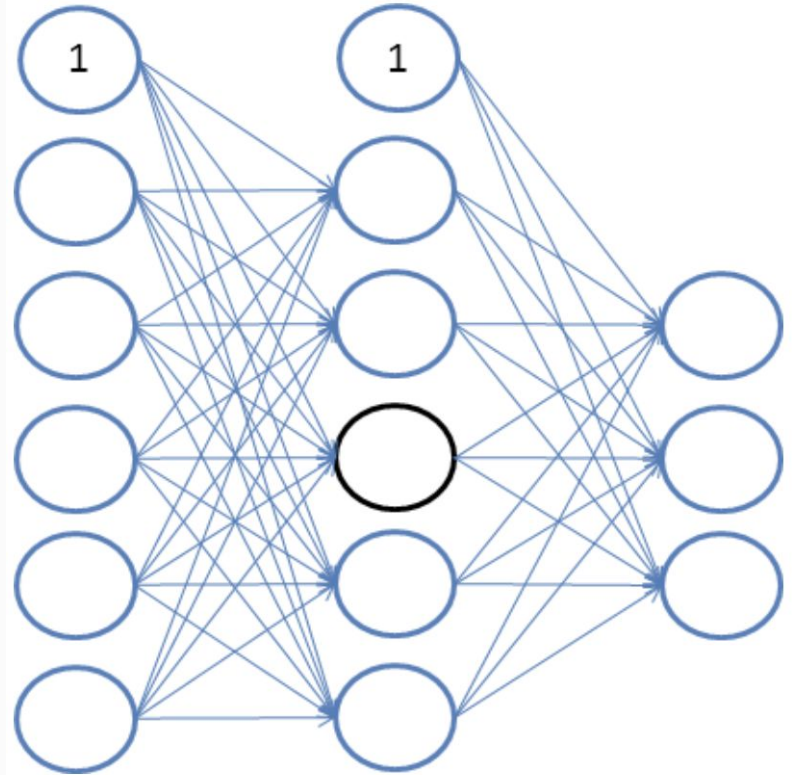
Activations

- **Input layer:** $a_i^0 = x_i$ (directly from input data)
- **Other layers:** $a_i^j = g(z_i^j)$, where g is the activation function (e.g., logistic), and z_i^j is the weighted sum of activations from the previous layer.

Example Notation for Nodes/Activation Units

Notation

- $a^j \rightarrow$ vector of activations for layer j
- $a_i^j \rightarrow$ activation of unit i in layer j
- Examples:
 - $a^0 = x \rightarrow$ input layer activations
 - $a^{L-1} = y \rightarrow$ output layer activations
 - $a^1 \rightarrow$ first hidden layer activations
 - $a_0^1 \rightarrow$ bias unit in first hidden layer (always 1)
 - $a_3^1 \rightarrow$ 3rd non-bias neuron in first hidden layer



No z (*logit*) Exist for the Input Layer

- The input layer does not compute anything
- It does not multiply by weights
- It does not apply an activation function
- The input layer just holds the raw data.

- Hidden + output layers:

$$z_i^j \rightarrow a_i^j$$

- Input layer:

$$a^0 = x \text{ directly}$$

Notation for Weights

- Edges in the network!
- We associate weights with the source layer (the layer they emerge from).
- Therefore every layer except the output layer has a set of weights.

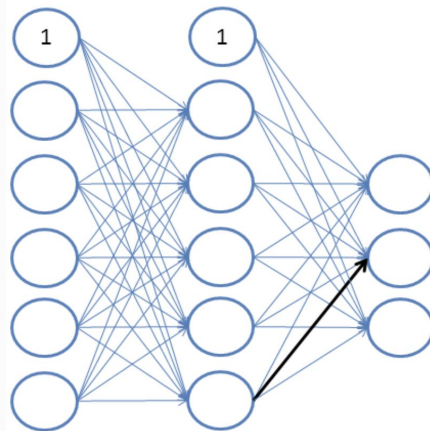
- θ^0 refers to the weights emerging from the input layer
- θ^{L-2} refers to the layer of weights feeding in to the output layer
- θ^1 refers to the weights emerging from the (first) hidden layer
- $\theta_{0,1}^1$ refers to the weight from the bias unit in the (first) hidden layer to the first unit in the following layer
- $\theta_{5,1}^1$ refers to the weight highlighted in the figure below.

Weights are denoted by: θ

Could be denoted with "w" also

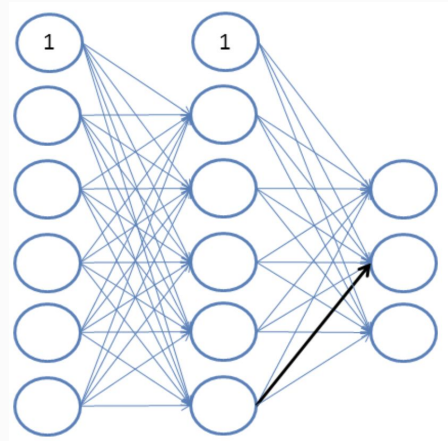
Indexing Rules

- **Superscript** $\rightarrow$ layer index
- **Subscript (k, i)** $\rightarrow$
 - First index = **source node** $\theta_{k,i}^j$
 - Second index = **target node**



Weight for Bias Units

- Bias units
 - Not computed from previous layers
 - Not the result of a weighted sum
 - Not passed through an activation function
 - No weights feed into bias units.
- Therefore, no weight can have target index 0!



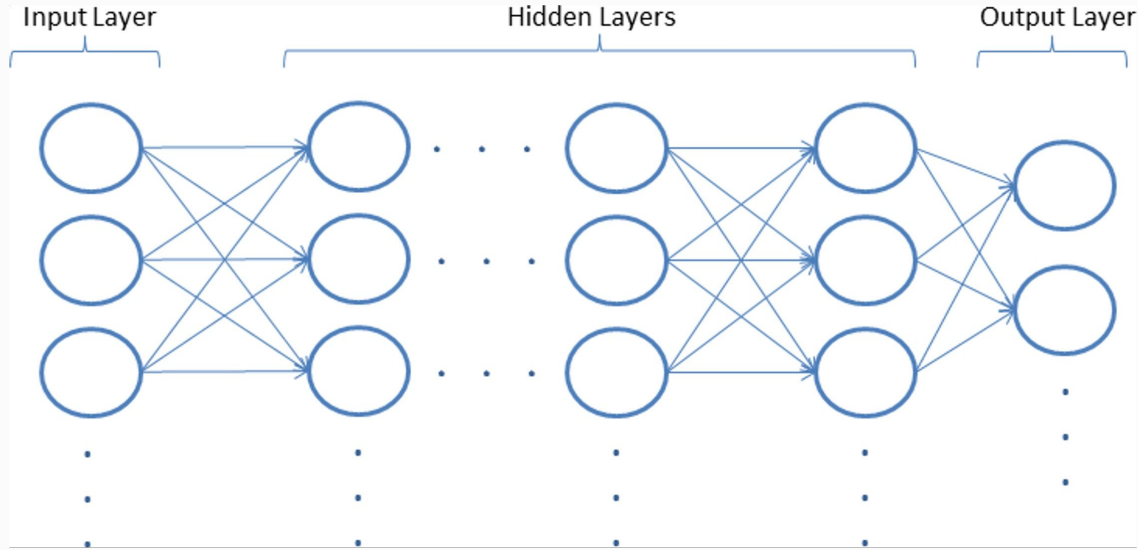
- The **target index can never be 0**
 - Because bias units are fixed at 1
 - They are not computed neurons
- The output layer has **no outgoing weights**

So:

$$\theta^{L-1} \quad \text{and} \quad \theta^L$$

are **invalid indices**.

Deep Feedforward Networks



- Deep Feedforward Neural Networks are simply networks that have more than one hidden layer.
- our networks will in practice have many more nodes at each layer and a number of hidden layers.

How Layers and Neurons Affect Model Capacity??

How Layers and Neurons Affect Model Capacity

Number of Neurons in a Layer (Width):

- More neurons → more parameters → the layer can represent **more complex functions**.
- Too few neurons → model may **underfit** (cannot capture patterns).
- Too many neurons → risk of **overfitting** (fits noise in training data).

Number of Layers (Depth):

- More layers → network can represent **hierarchical or compositional features**.
- Deeper networks can approximate **more complex functions** than shallow ones with the same total number of neurons.
- Each layer can transform features into a **higher-level representation**.

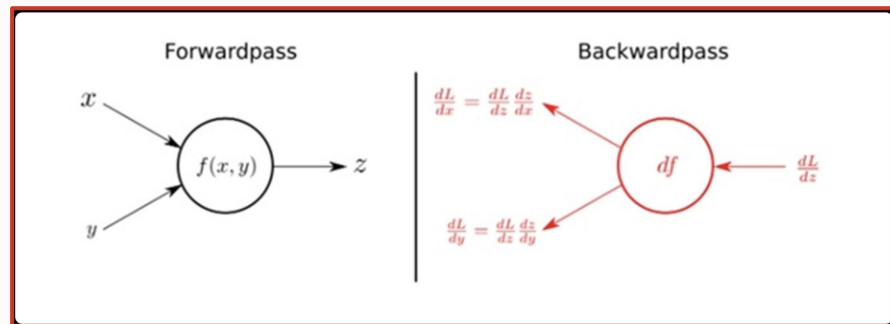
Expressiveness of a network

- Width (neurons per layer) controls how much detail each layer can capture, while depth (number of layers) allows the network to combine features hierarchically, giving it greater overall expressiveness.

Two Key Algorithms in Neural Networks & Deep Learning

- **Forward Propagation (Feedforward Algorithm)**

- Purpose: Compute networks current predictions for the input
- To do so, pass information layer by layer
- Used in training and inference
- Does not change weights



- **Backward Propagation Algorithm**

- Purpose: Determine how to adjust the weights to reduce prediction error.
- Calculate gradients and provide direction for weight updates

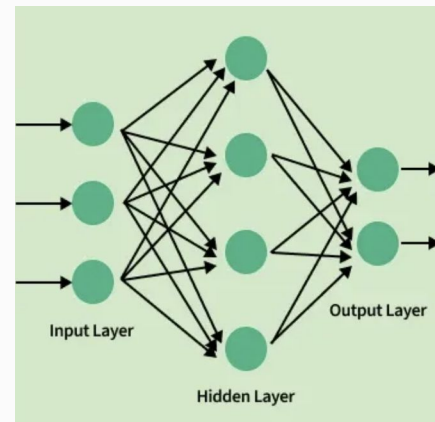
Forward pass answers: “What does the network predict?”

Backward pass answers: “How should we change the weights to improve the prediction?”

Feedforward Algorithm — Layer-by-Layer Process

For Each Layer (From First Hidden Layer to Output Layer)

1. Compute the Weighted Sum (Pre-Activation)
2. Apply the Activation Function
3. Pass Output to the Next Layer



Feedforward Algorithm — Layer-by-Layer Process

For Each Layer (From First Hidden Layer to Output Layer)

1. Compute the Weighted Sum (Pre-Activation)
2. Apply the Activation Function
3. Pass Output to the Next Layer

Special Case: First Hidden Layer

For the first hidden layer:

- Input comes directly from the data:

No logit/activation!

$$a^0 = x$$

Special Case: Output Layer

For the output layer:

- Compute weighted sum
- Apply activation function
- Produce final prediction

There is no next layer, so the result is:

$$\hat{y} = a^{L-1}$$

Returned as the model's output.

Feedforward Algorithm

For each layer:

1. Compute weighted sum (including bias)
2. Apply activation function
3. Pass result to next layer

Final layer returns prediction.

Calculating Cost / Error in Neural Networks

Once the input has passed through network via feedforward algorithm, we can measure how well network predicted the output with calculating the **cost**.

Cost Calculation

- Analogous to linear regression or logistic regression
- Measures the difference between predicted output and true target
- Forms the basis for learning and weight updates

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

Cross-entropy (log-loss)

- Output Layer with Logistic / Sigmoid Activation
- Suitable for binary classification
- Measures prediction probability error

$$\text{Cost} = J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Loss}(\hat{y}^{(i)}, y^{(i)})$$

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

Mean squared error (MSE)

- Suitable for regression tasks
- Cost function measures direct difference between predicted and true values

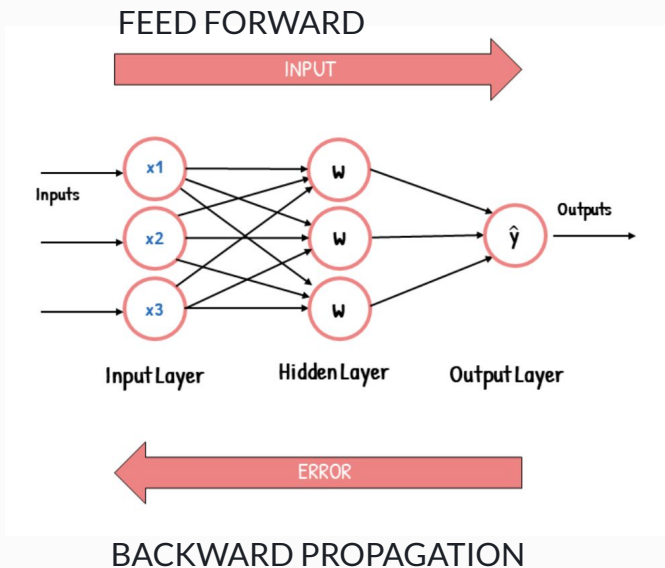
Output Layer Activation Depends on the Task

Examples

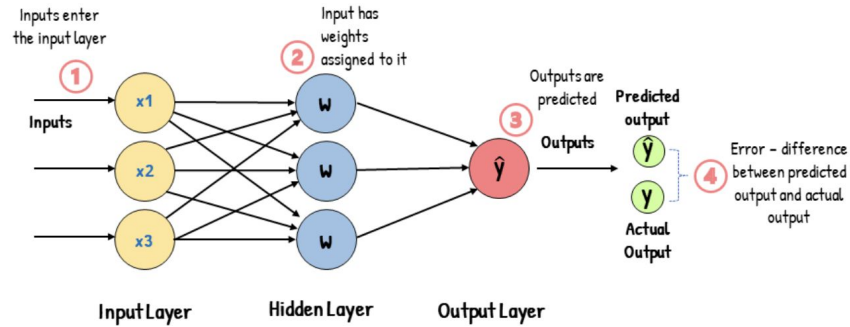
Task Type	Common Output Activation	Notes
Binary classification	Sigmoid (logistic)	Produces a probability between 0 and 1
Multi-class classification	Softmax	Produces probabilities over multiple classes
Regression (real-valued)	None / Linear	Outputs any real number; no squashing needed

Once the cost is calculated, it is used in **backpropagation** to update weights.

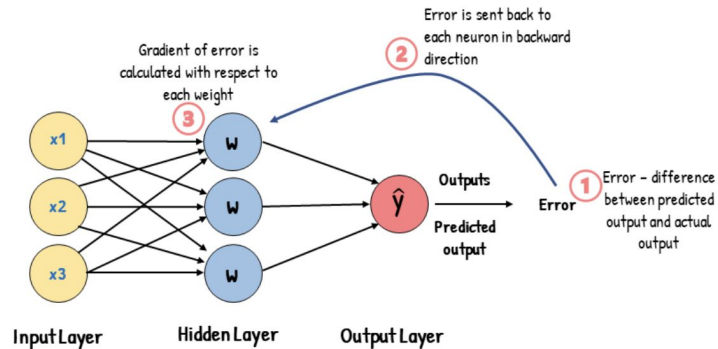
- In linear and logistic regression, cost function is convex → has a single layer, gradient is simple to compute.
- In neural networks
 - Multiple layers + non-linear activations → cost is highly non-linear and non-convex.
 - no simple formula gives the best weight updates directly
- Backpropagation Algorithm
 - computes gradients of the cost with respect to all weights using the chain rule of calculus
 - these gradients are then used by an optimization algorithm (like gradient descent) to iteratively update the weights and reduce the cost.



Feed-Forward



Backpropagation



TensorFlow and Keras for Deep Learning

If we coded neural networks manually, we would need to:

- Initialize weights
- Perform matrix multiplications
- Apply activation functions
- Compute loss
- Compute gradients
- Update weights
- Repeat thousands of times

This quickly becomes complicated. Deep learning frameworks automate this process.

What is TensorFlow and Keras?

TensorFlow is a machine learning framework developed by Google.

It helps us:

- Perform fast numerical computations
- Train deep neural networks
- Automatically compute gradients
- Use GPUs for acceleration
- **Think of TensorFlow as: A powerful engine for training neural networks.**

Keras is a high-level interface built on top of TensorFlow.

It allows us to:

- Build neural networks with very little code
- Focus on architecture rather than mathematics
- Experiment quickly
- Today we use: **tf.keras**
- Keras is now fully integrated into TensorFlow.

Tensorflow

Coding

When you write `model.fit(X_train, y_train, epochs=20)`

TensorFlow automatically performs:

1. Forward propagation: Computes outputs layer by layer
2. Loss calculation: Compares predictions with true labels
3. Backpropagation: Uses automatic differentiation to compute gradients
4. Weight updates: Applies the chosen optimizer (e.g., Adam, SGD)
5. Repeats for each batch and epoch

You do not manually:

- Compute derivatives
- Apply the chain rule
- Update each weight individually

TensorFlow = The Engine

Keras = The User-Friendly Dashboard

You design the network in Keras.

TensorFlow performs:

- Forward propagation
- Backpropagation
- Gradient calculations
- Weight updates

Frameworks automate mechanics, understanding mechanics help you troubleshoot training issues, choose appropriate architectures, and improve performance.



The Basic Deep Learning Workflow

When training a model we:

1. Define the model architecture
2. Choose a loss function
3. Choose an optimizer
4. Train the model
5. Evaluate performance

TensorFlow/Keras manages the mechanics behind the scenes.

What is a Tensor?

- A **tensor** is just a generalization of familiar concepts like **scalars, vectors, and matrices** to potentially higher dimensions.
- Think of it as a container for numbers that can have **any number of dimensions**.
- In deep learning we always work with numbers arranged in structures/tensors:

Dimension	Name	Example	Shape
0D Tensor	Scalar	5	()
1D Tensor	Vector	[1, 2, 3]	(3,)
2D Tensor	Matrix	[[1,2], [3,4]]	(2,2)
3D Tensor	Stack of matrices	RGB Image	(height, width, channels)

Building a Simple Network in Keras

Coding

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

You specify:

- Number of layers
 - Number of neurons
 - Activation functions
- Layers: 1 hidden layer + 1 output layer (input layer is implicit, not counted)
 - Hidden Layer: 16 neurons, activation = ReLU
 - Output Layer: 1 neuron, activation = Sigmoid
 - Bias Units: Not counted as neurons, fixed value = 1, no activation
 - Dense Layer: Fully connected to previous layer
 - Every neuron in the current layer receives input from **all neurons in the previous layer**.
 - Model Type: **Sequential** (layers stacked one after another)
 - There are other types such as Functional API which allows skipping connections, branching or merging layers.

You do NOT manually:

- Initialize weights
- Write backpropagation
- Compute gradients

Build Functional Model

Used for building complex neural networks with flexible data flow (branches, merges, multiple inputs/outputs)

```
# Apply layers independently to each input
```

```
x1 = Dense(8, activation='relu')(input1)
```

```
x2 = Dense(8, activation='relu')(input2)
```

```
# Merge the two branches
```

```
combined = Concatenate()([x1, x2])
```

```
# Output layer
```

```
output = Dense(1, activation='sigmoid')(combined)
```

```
# Create the model
```

```
model = Model(inputs=[input1, input2], outputs=output)
```

Use Sequential for simple, straight-forward models.

Use Functional API whenever you need branches, merges, multiple inputs/outputs, or skip connections.

- Two inputs → processed separately → merged → output.
 - Ex: a network that takes both image data and metadata (e.g., age, location).
 - Each input can be processed differently before merging.
- Layers act as functions: `output = Layer(input_tensor)`.
- Flexible for multi-input, multi-output, or branched networks.
- Cannot do this in a Sequential model.

reference

Nice Video Showing Feedforward Network

<https://www.youtube.com/watch?v=llg3gGewQ5U>

Lab Work

Please follow the instructions in the Week-4 notebook in the last cell to build a notebook from scratch and play with some vector operations.

Project Proposal Due on 25th of Feb
First Assessment will be on 11th of March

APPENDIX